REPORT OF ADVANCED MACHINE LEARNING PROJECT

Submitted by Blessen Koshy Sujan (23645007)

1. Data Processing for Machine Learning

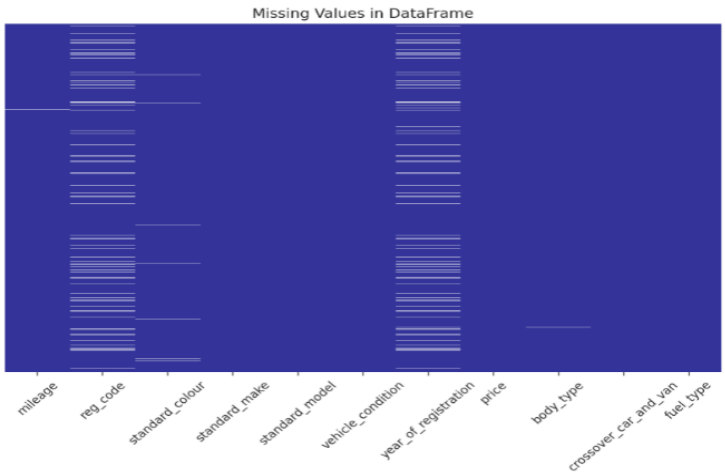
- The car trader dataset had 402005 data points which had also 12 columns which was treated as the features of the dataset.
- The features were split into categorical and numerical features. **cd.describe** was executed to have closer look at how the numerical features were distributed in set.

```
cd.head(4)
```

	public_reference	mileage	reg_code	standard_colour	standard_make	standard_model	vehicle_condition	year_of_registration	price	body_type	crossover_car_and_van	fuel_type
0	202006039777689	0.0	NaN	Grey	Volvo	XC90	NEW	NaN	73970	SUV	False	Petrol Plug-in Hybrid
1	202007020778260	108230.0	61	Blue	Jaguar	XF	USED	2011.0	7000	Saloon	False	Diesel
2	202007020778474	7800.0	17	Grey	SKODA	Yeti	USED	2017.0	14000	SUV	False	Petrol

```
cd.isna().sum()
```

```
mileage                  127
reg_code                31857
standard_colour          5378
standard_make               0
standard_model              0
vehicle_condition           0
year_of_registration    33311
price                       0
body_type                 837
crossover_car_and_van       0
fuel_type                 601
dtype: int64
```



- The missing values in mileage was countered by replacing the NaN values with mean values whereas, the NaN values in standard colour, body type and fuel type were replaced by the mode in the specific column.

```
mean_mileage=cd['mileage'].mean()
```

```
cd['mileage'].fillna(mean_mileage, inplace=True)
```

```
mode_color = cd['standard_colour'].mode()[0]
```

```
cd['standard_colour'].fillna(mode_color,inplace= True)
```

```
mode_color = cd['body_type'].mode()[0]
cd['body_type'].fillna(mode_color,inplace= True)
mode_color = cd['fuel_type'].mode()[0]
cd['fuel_type'].fillna(mode_color,inplace= True)
```

```
cd.loc[cd['vehicle_condition']=='NEW','year_of_registration']= 2021
```

- The 31249 values in year of registration were found in the vehicle condition 'NEW', it was assumed and to be registered in in the year 2021 as the latest year in dataset was 2020.
- The remaining 2062 values were found in column with vehicle condition 'USED' whose registration year and reg code were missing and was removed.
- According to details found, the vehicle registration started in 1904 so all the vehicles that has registration year less than 1904 (17) were also removed.
- To deal with the outliers in numerical features, IQR was done.

```
Q1 = cd['price'].quantile(0.25)
Q3 = cd['price'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outlier_mask = (cd['price'] >= lower_bound) & (cd['price'] <= upper_bound)
cd =cd[outlier_mask]
```

```
Q1 = cd['mileage'].quantile(0.25)
Q3 = cd['mileage'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outlier_mask = (cd['mileage'] >= lower_bound) & (cd['mileage'] <= upper_bound)
cd =cd[outlier_mask]
```

```
standard_make                  0
standard_model                 0
standard_colour                0
body_type                      0
fuel_type                      0
mileage                        0
price                          0
Age                            0
Avg_Mile_per_year              0
crossover_car_and_van_False    0
crossover_car_and_van_True     0
vehicle_condition_NEW          0
vehicle_condition_USED         0
dtype: int64
```

- Cross over car and van and vehicle conditon was one hot encoded and the remaining features were target encoded with the target variable ‘price’.
- Later, dataset was split into target Y and features into X, which was futher split into train and test set and standard scaling was done.

```
from sklearn.model_selection import train_test_split
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=0)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=0)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

2. Feature Engineering

- For featuring, a new feature ‘Age’ had been created, by subtracting the year of registration from the current year 2024 and the existing column ‘year of registration’ was removed.
- Additionally, the ratio of mileage vs Age was created as an interaction feature to visualize how mileage varies with Age and how it will impact our model.

```
cd['Age']= 2024- cd['year_of_registration']
```

```
cd['Avg_Mile_per_year'] = cd['mileage']/cd['Age']
```

- The reg code and public reference were dropped as the columns didn’t contribute much for the prediction of the price as required for our dataset.

```
cd=cd.drop('reg_code','public_reference' axis =1)
```

- Cross over car and van and vehicle condition columns were one hot encoded to know how it could impact the model.

```
cde = pd.get_dummies(cd, columns=['crossover_car_and_van'])
cde.head(3)
```

```
cde1 = pd.get_dummies(cde, columns=['vehicle_condition'])
cde1.head(3)
```

- Then to visualize, whether the higher order of numerical features such as Age and Mileage contribute to model, polynomial features were introduced and was generated as Poly Age, Poly Mileage and Poly Age Mileage.
- Then a polynomial regression is fitted to see its impact on the model.

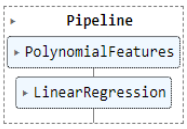
```
poly_features=['Age','mileage']
```

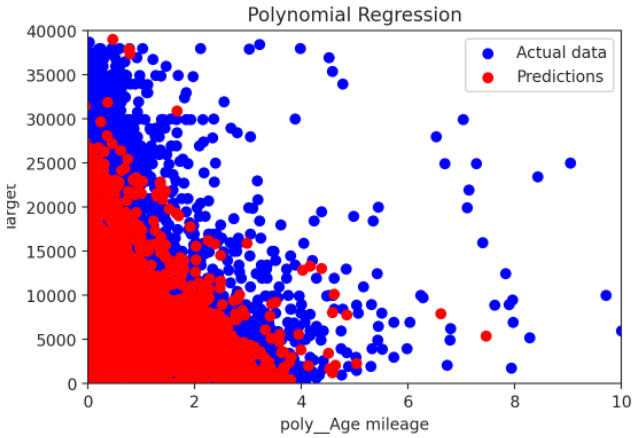
```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.compose import ColumnTransformer
```

```
poly_pipeline= Pipeline([
    ('poly',PolynomialFeatures(degree=2, include_bias= False, interaction_only= True))
])
```

```
preprocessor = ColumnTransformer(
                    transformers=[
                        ('poly', poly_pipeline, poly_features)
```

```
degree = 2
model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
model.fit(X_train, y_train)
```





A visualization of how poly age mileage varies with target variable is shown with its actual data and predictions and it can be observed that most of the points do not coincide indicating the polynomial feature cannot contribute much to the model.

- After encoding, the difference between values in feature very huge, so MinMax scaling was done to maintain all in data values in a similar scale.

```
numeric_features = X_train.select_dtypes(exclude='object').columns.tolist()
numeric_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("scaler", MinMaxScaler()),
    ]
)

print(numeric_features)
numeric_transformer
```

```
['standard_make', 'standard_model', 'standard_colour', 'body_type', 'fuel_type',
Pipeline
SimpleImputer
MinMaxScaler
```

```
X_train=numeric_transformer.fit_transform(X_train[numeric_features])
```

standard_make	standard_model	standard_colour	body_type	fuel_type	mileage	Age	Avg_Mile_per_year	crossover_car_and_van_False
0.453375	0.513537	0.700181	0.717178	0.126176	0.574581	0.008806	0.195155	1.0
0.178949	0.191742	0.700181	0.055482	0.126176	0.211426	0.004892	0.107716	1.0
0.200087	0.210199	0.700181	0.000000	0.000000	0.000000	0.000000	0.000000	1.0
0.269371	0.261669	0.404783	0.717178	0.000000	0.173450	0.004892	0.088367	1.0
0.453375	0.337872	0.700181	0.000000	0.126176	0.217054	0.003914	0.126380	1.0

3. Feature Selection and Dimensionality Reduction

- Reducing the number of features can improve model performance, mitigate overfitting, and remove redundant or noisy information, leading to more accurate and robust results.
- Three feature selection techniques such as KBest, RFECV and SFS were performed on all models and were evaluated with cross validation to reduce the number of features.
- Initially, the dataset produced after encoding had 15 columns each representing a particular feature.
- The best performing model was XGB regressor in all three above mentioned automated systems.

```
xgb_bestk.fit(X_train, y_train)
```

```
Pipeline
SelectKBest
XGBRegressor
```

```
y_pred = xgb_bestk.predict(X_test)
```

```
xgb_bestk['featsel'].get_feature_names_out()
```

```
array(['standard_make', 'standard_model', 'body_type', 'mileage',
       'Avg_Mile_per_year', 'poly__mileage', 'poly__Age mileage'],
      dtype=object)
```

```
xgb_regr_rfe = Pipeline(
    steps=[
        #("preprocessor", preprocessor),
        ("featsel", RFECV(xgb_regressor, step=1, cv=5)),
        ("regr", xgb_regressor)
    ]
)
```

```
xgb_regr_rfe.fit(X_train, y_train)
```

```
Pipeline
featsel: RFECV
estimator: XGBRegressor
XGBRegressor
XGBRegressor
```

```
y_pred = xgb_regr_rfe['featsel'].predict(X_test)
```

```
xgb_regr_rfe['featsel'].get_feature_names_out()
```

```
array(['standard_make', 'standard_model', 'body_type', 'fuel_type', 'Age',
       'crossover_car_and_van_False', 'poly__Age mileage'], dtype=object)
```

- The features that came out as top 6 were **Standard make, standard model, body type, fuel type, Age and Mileage**
- The score of RFECV with feature selection was higher with 0.92 in test and 0.94 in train set.

```
xgb_regr_rfe['featsel'].score(X_test, y_test), xgb_regr_rfe['featsel'].score(X_train, y_train)
```

```
(0.9263399795597651, 0.9409346470548094)
```

```
mean_absolute_error(y_test, xgb_regr_rfe['featsel'].predict(X_test))
```

```
1529.9340119162546
```

```
sfs_forward_xgb = SequentialFeatureSelector(
  xgb_regressor, n_features_to_select='auto', direction="forward")
```

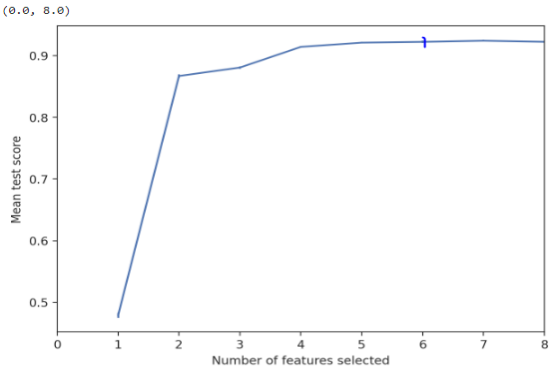
```
sfs_forward_xgb.fit(X_train, y_train)
```

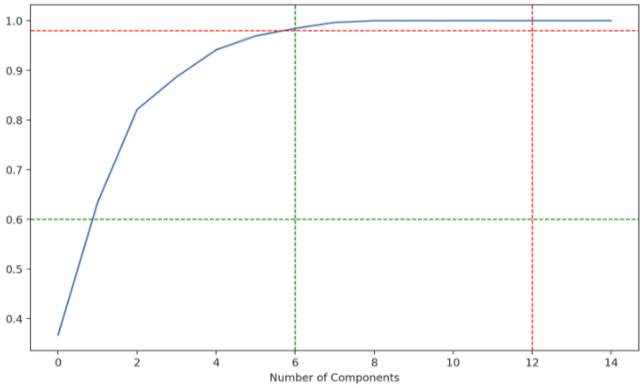
```
SequentialFeatureSelector
estimator: XGBRegressor
XGBRegressor
```

```
sfs_forward_xgb.get_feature_names_out()
```

```
array(['standard_make', 'standard_model', 'body_type', 'fuel_type', 'Age',
       'crossover_car_and_van_False', 'poly__Age mileage'], dtype=object)
```

- The polynomial features didn't contribute much to model, so they were dropped.
- Even though, cross over car and van came ahead of Mileage, mileage was chosen on basis of domain knowledge and a proper definition of cross over car and van were missing and didn't match with body type. E.g. body type SUV was found in both cross over car and van TRUE and FALSE
- The XGB model with feature selection gave us almost similar score on both test and train sets compared to other models with AFS and the mean absolute score was also lower in XGB.
- PCA was tested to reduce the dimensionality of the dataset. It was found that with just 6 features, the variance reached up to 0.98 it means that 98% of the total variability in the dataset can be captured by a reduced set of dimensions of Six and all the remaining features contributes only 2%.





- In the first graph by RFCEV with XGB, after the number of features reached six, the mean test score remained constant which is supported by the variance graph obtained using PCA.
- The features that were taken for model building are **Standard make, standard model, body type, fuel type, Age and Mileage** taking into consideration of feature importance of individual models and all other features including the polynomial features were removed from Xtest and Xtrain dataset.

4. Model Building

4.1 Linear Regression

It is a statistical method used to model the relationship between a dependent variable and one or more independent variables by fitting a linear equation to the observed data. Its goal is to find the line that best minimizes the difference between the actual and predicted values, allowing for predictions or inferences about the dependent variable based on the independent variables.

```
lr = LinearRegression()
```

```
lr.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
lr_predictions = lr.predict(X_test)
```

```
lr_rmse = mean_squared_error(y_test, lr_predictions, squared=False)
lr_rmse
```

```
4181.927961166673
```

```
scores_lr = cross_val_score(lr, X_train, y_train, cv=5)
scores_lr
```

```
array([0.73994919, 0.03954085, 0.74823914, 0.7386372 , 0.742614  ])
```

```
scores_lr1 = cross_val_score(lr, X_test, y_test, cv=5)
scores_lr1
```

```
array([0.77708342, 0.76221042, 0.75590624, 0.76642442, 0.76009775])
```

- The linear regression model was fitted and the RMSE value of it was found to be 4181.92.
- The cross-validation scores of linear regression model among the five folds ranged between 0.73 and 0.74 in train set and in test set, it varied between 0.75 and 0.77.

```
mean_absolute_error(y_train, lr.predict(X_train))
```

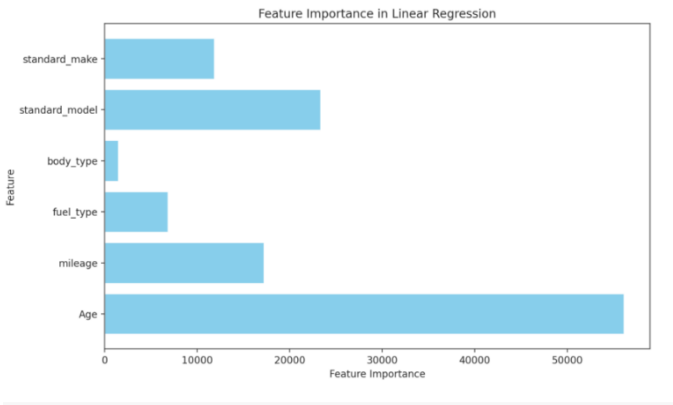
```
3160.8705588905273
```

```
mean_absolute_error(y_test, lr.predict(X_test))
```

```
3154.9832289438264
```

```
r2_lr = r2_score(y_test, lr_predictions)
r2_lr
```

```
0.7469224213749746
```



- The Mean Absolute error in train set was 3160.87 and in test set was 3154.98. Their closeness indicates the model works almost similar in both sets and the r2 score of the model is 0.74 which gave insights that higher the value of r2, better the model works.
- The feature importance graph of the model is also shown, and this model predict higher feature value for Age and lower for body type.
- The feature importance order of linear regression was Age, standard model, mileage, standard make, fuel type and body type.
- Since, there are no hyper parameters of LR, grid search is not done.

4.2 Random Forest Regressor

Random Forest regression is an ensemble learning method that constructs multiple decision trees during training and outputs the average prediction of the individual trees. It's effective for capturing complex relationships in data and tends to be more robust against overfitting compared to a single decision tree.

- The Random Forest regression model was fitted and the RMSE value of it was found to be 2369.04.
- The cross-validation scores of random forest regression model among the five folds ranged between 0.911 and 0.915 in train set and in test set, it varied between 0.88 and 0.90.

```
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
```

```
rf_regressor = RandomForestRegressor(n_estimators=300, random_state=42)
```

```
rf_regressor.fit(X_train, y_train)
```

```
RandomForestRegressor
RandomForestRegressor(n_estimators=300, random_state=42)
```

```
param_grid = {
    'n_estimators': [100, 200, 300],
}
```

```
grid_search = GridSearchCV(estimator=rf_regressor, param_grid=param_grid, cv=5, n_jobs=-1, verbose=2)
```

```
grid_search.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 3 candidates, totalling 15 fits
GridSearchCV
estimator: RandomForestRegressor
RandomForestRegressor
```

```
print("Best hyperparameters:", grid_search.best_params_)
```

- Grid search was done on RF and was found that the best hyperparameter for the model was n_estimators = 300.

```
rf_regressor_rmse = mean_squared_error(y_test, rf_regressor_predictions, squared=False)
rf_regressor_rmse
```

```
2399.049250203758
```

```
scores_rf = cross_val_score(rf_regressor, X_train, y_train, cv=5)
scores_rf
```

```
array([0.91195482, 0.91451266, 0.91968858, 0.9157867 , 0.91438707])
```

```
scores_rf1 = cross_val_score(rf_regressor, X_test, y_test, cv=5)
scores_rf1
```

```
array([0.90056493, 0.88621609, 0.90481304, 0.90403455, 0.89200913])
```

```
mean_absolute_error(y_train, rf_regressor.predict(X_train))
```

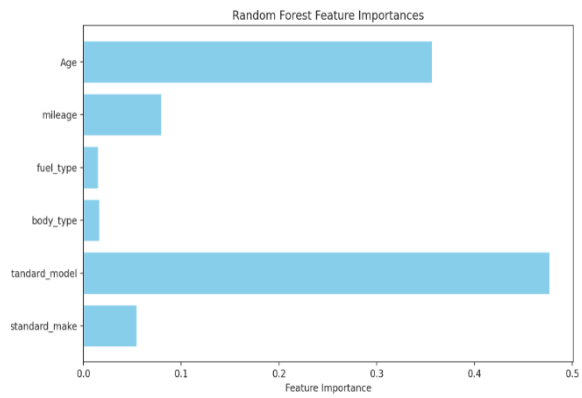
```
640.7053695658641
```

```
mean_absolute_error(y_test, rf_regressor.predict(X_test))
```

```
1581.9724545487622
```

```
r2_rf = r2_score(y_test, rf_regressor_predictions)
r2_rf
```

```
0.9167126769777075
```



- The mean absolute error in train set of the model was 640.70 and in test set it increased to 1581.97. The r squared value of the model was 0.9167.
- Comparing to the previous lr model, the rf model could predict accurately about 91.67%
- The feature importance of the model was in the order: Model, Age, Mileage, Make, Body type and Fuel type.

4.3.1 XGB Regressor

XGBoost regressor is an advanced implementation of gradient boosting algorithms, known for its efficiency and high performance in regression tasks by sequentially improving the predictions of weak learners. It constructs multiple decision trees iteratively, optimizing a specified objective function, and combines their predictions to produce a final output.

- The XGB regression model was fitted and the RMSE value of it was found to be 2261.49.

```
import xgboost as xgb
```

```
xgb_regressor = xgb.XGBRegressor()
```

```
xgb_regressor.fit(X_train, y_train)
```

```
XGBRegressor
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             gamma=None, grow_policy=None, importance_type=None,
             interaction_constraints=None, learning_rate=None, max_bin=None,
             max_cat_threshold=None, max_cat_to_onehot=None,
             max_delta_step=None, max_depth=None, max_leaves=None,
             min_child_weight=None, missing=nan, monotone_constraints=None,
             multi_strategy=None, n_estimators=None, n_jobs=None,
             num_parallel_tree=None, random_state=None, ...)
```

```
param_grid = {
    'max_depth': [3,5,7],
    'learning_rate': [0.1, 0.01],
    'n_estimators': [100, 200],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}
```

```
grid_search = GridSearchCV(estimator=xgb_regressor, param_grid=param_grid, cv=3, scoring='accuracy', verbose=1, n_jobs=-1)
```

```
grid_search.fit(X_train, y_train)
```

```
Fitting 3 folds for each of 48 candidates, totalling 144 fits
GridSearchCV
estimator: XGBRegressor
XGBRegressor
```

```
best_params = grid_search.best_params_
print("Best Hyperparameters:", best_params)
```

```
Best Hyperparameters: {'colsample_bytree': 0.8, 'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 100, 'subsample': 0.8}
```

- The cross-validation scores of XGB regression model among the five folds ranged between 0.922 and 0.925 in train set and in test set, it varied between 0.89 and 0.914

```
xgb_regressor_rmse = mean_squared_error(y_test, xgb_regressor_predictions, squared=False)
xgb_regressor_rmse
```

```
2261.499797043016
```

```
scores_xgb = cross_val_score(xgb_regressor, X_train, y_train, cv=5)
scores_xgb
```

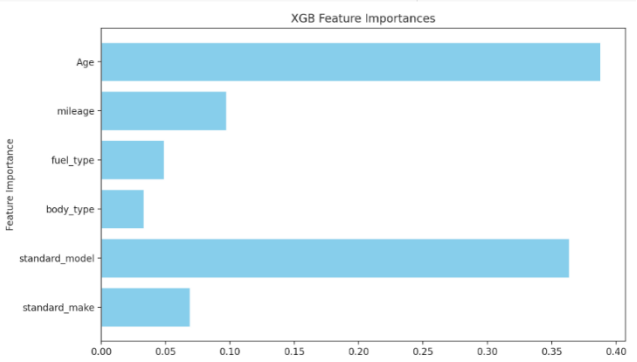
```
array([0.92209637, 0.92429854, 0.92542241, 0.92366733, 0.9229351 ])
```

```
scores_xgb1 = cross_val_score(xgb_regressor, X_test, y_test, cv=5)
scores_xgb1
```

```
array([0.91075867, 0.89734819, 0.91148088, 0.91450597, 0.90802345])
```

```
r2_xgb = r2_score(y_test, xgb_regressor_predictions)
r2_xgb
```

```
0.9259894413762949
```



- Grid search was done on XGB and was found that the best hyperparameter for the model was n_estimators = 100, learning rate=0.1, max depth=3, subsample and col sample by tree as 0.8.
- The mean absolute error in train set of the model was 1541.80 and in test set it decreased to 1379.34. The r squared value of the model was 0.9259.
- The feature importance of the model was in the order: Age, Model, Mileage, Make, Fuel type and Body type.
- This is the best model of all the model test when evaluation metrics are considered.

4.3.2 Gradient Boosting Regressor

Gradient Boosting (GB) regressor is a powerful machine learning technique that sequentially builds a series of weak regression models and combines them to form a strong ensemble model, minimizing the residual errors at each step. By iteratively optimizing a predefined loss function, GB regressor effectively captures complex nonlinear relationships in the data, making it well-suited for regression tasks where high predictive accuracy is paramount.

- The GB regression model was fitted and the RMSE value of it was found to be 2999.50.
- The cross-validation scores of GB regression model among the five folds ranged between 0.865 and 0.869 in train set and in test set, it varied between 0.862 and 0.866.
- The Mean Absolute error in train set was 2046.25 and in test set was 2074.49.

```
from sklearn.ensemble import GradientBoostingRegressor
```

```
gb_regressor = GradientBoostingRegressor(n_estimators=100, random_state=42)
```

```
gb_regressor.fit(X_train, y_train)
```

```
GradientBoostingRegressor
GradientBoostingRegressor(random_state=42)
```

```
gb_regressor_predictions = gb_regressor.predict(X_test)
```

```
gb_regressor_rmse = mean_squared_error(y_test, gb_regressor_predictions, squared=False)
gb_regressor_rmse
```

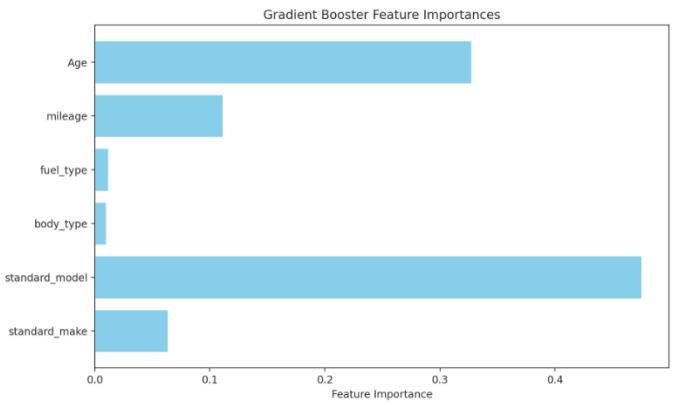
```
2999.5086121948866
```

```
mean_absolute_error(y_train, gb_regressor.predict(X_train))
```

```
2046.2579634447443
```

```
mean_absolute_error(y_test, gb_regressor.predict(X_test))
```

```
2074.4997224940244
```



The feature importance of the model was in the order: Model, Age, Mileage, Make, Fuel type and Body type.

The r2 score of the model was 0.8698.

4.4 Voting Model Ensemble

A Voting Ensemble is a machine learning technique that combines the predictions from multiple individual models to make a final prediction. It works by aggregating the predictions of each base model and selecting the most common prediction (for classification tasks) or averaging the predictions (for regression tasks). Voting ensembles can improve predictive performance by leveraging the strengths of different models and mitigating their individual weaknesses, resulting in a more robust and accurate final prediction.

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import VotingRegressor
from sklearn.ensemble import StackingRegressor
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import mean_squared_error
```

```
model1 = RandomForestRegressor(n_estimators=100, random_state=42)
model2 = xgb.XGBRegressor()
model3= GradientBoostingRegressor(n_estimators=100, random_state=42)
```

```
model1.fit(X_train, y_train)
```

```
RandomForestRegressor
RandomForestRegressor(random_state=42)
```

```
model2.fit(X_train, y_train)
```

```
XGBRegressor
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             gamma=None, grow_policy=None, importance_type=None,
             interaction_constraints=None, learning_rate=None, max_bin=None,
             max_cat_threshold=None, max_cat_to_onehot=None,
             max_delta_step=None, max_depth=None, max_leaves=None,
             min_child_weight=None, missing=nan, monotone_constraints=None,
             multi_strategy=None, n_estimators=None, n_jobs=None,
             num_parallel_tree=None, random_state=None, ...)
```

```
model3.fit(X_train, y_train)
```

```
GradientBoostingRegressor
GradientBoostingRegressor(random_state=42)
```

- A voting model was built with best performing models such random forest, XGB regressor and GB regressor. Linear regression was opted out as it was under performing compared to these three models.

Voting

```
[ ] voting_model.fit(X_train, y_train)
```

```
VotingRegressor
rf: RandomForestRegressor
xgb: XGBRegressor
gb: GradientBoostingRegressor
```

```
[ ] voting_pred = voting_model.predict(X_test)
```

- The RMSE of the model was 2358.59 which was second to the XGB regressor value. The mean absolute error in test set was 1604.43 and in train it was 1261.78.
- The r2 score of the model was 0.9194 and the voting model had consistent performance on the training set and test set.

```
eval_resultsv = cross_validate(
    voting_model, X_train, y_train, cv=5,
    scoring='neg_mean_absolute_error',
    return_train_score=True
)
(
    (-eval_resultsv['test_score'].mean(), eval_resultsv['test_score'].std(),
    -eval_resultsv['train_score'].mean(), eval_resultsv['train_score'].std())
)
```

```
(1604.4340459833368, 22.220034009102125, 1261.7859151352654, 5.619753504538032)
```

```
voting_model_rmse = mean_squared_error(y_test, voting_pred, squared=False)
voting_model_rmse
```

```
2358.59329941285
```

```
scores_voting = cross_val_score(voting_model, X_train, y_train, cv=5)
scores_voting
```

```
array([0.91577852, 0.91709577, 0.9214187 , 0.91802885, 0.91677064])
```

```
scores_voting1 = cross_val_score(voting_model, X_test, y_test, cv=5)
scores_voting1
```

```
array([0.90520652, 0.89259642, 0.90850561, 0.91071641, 0.90035066])
```

```
from sklearn.metrics import r2_score
```

```
r2_voting = r2_score(y_test, voting_pred)
r2_voting
```

```
0.9194979950558947
```

Based on the evaluation metrics, the models were ranked according to their Root Mean squared value, the lower the value, the higher the performance of the model. And r2 score, closeness to 1 indicating better model.

RANKING ACCORDING TO RMSE
1.XGB REGRESSOR
2.VOTING MODEL
3. RF REGRESSOR
4.GB REGRESSOR
5. LINEAR REGRESSION

RANKING ACCORDING TO R2 SCORE
1.XGB REGRESSOR
2. RF REGRESSOR
3. VOTING MODEL
4.GB REGRESSOR
5. LINEAR REGRESSION

5. Model Evaluation and Analysis

5.1 Overall Performance with Cross-Validation

The top 3 models were XGB regressor, Rf regressor and Voting Ensemble. Their performance five folds were conducted to check the consistency in prediction of these models.

```
scores_xgb = cross_val_score(xgb_regressor, X_train, y_train, cv=5)
scores_xgb
```

```
array([0.92209637, 0.92429854, 0.92542241, 0.92366733, 0.9229351 ])
```

```
scores_xgb1 = cross_val_score(xgb_regressor, X_test, y_test, cv=5)
scores_xgb1
```

```
array([0.91075867, 0.89734819, 0.91148088, 0.91450597, 0.90802345])
```

```
r2_xgb = r2_score(y_test, xgb_regressor_predictions)
r2_xgb
```

```
0.9259894413762949
```

The XGB model has performed consistently in its test and train data set. Unseen test data had a slight lower value compared to the train data.

The train scores range from approximately 0.92 to 0.93, indicating that the model achieves a high level of accuracy on the training data across different folds. Overall, these cross-validation scores suggest that the XGBoost regressor generalizes well to both the training and testing data, as indicated by consistently high scores across different folds.

```
scores_rf = cross_val_score(rf_regressor, X_train, y_train, cv=5)
scores_rf
```

```
array([0.91195482, 0.91451266, 0.91968858, 0.9157867 , 0.91438707])
```

```
scores_rf1 = cross_val_score(rf_regressor, X_test, y_test, cv=5)
scores_rf1
```

```
array([0.90056493, 0.88621609, 0.90481304, 0.90403455, 0.89200913])
```

```
r2_rf = r2_score(y_test, rf_regressor_predictions)
r2_rf
```

```
0.9167126769777075
```

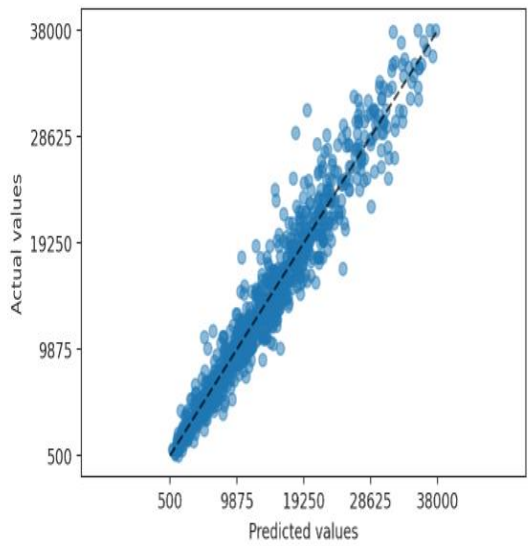
The rf model also has performed consistently in its test and train data set. Unseen test data had a slight lower value compared to the train data.

However, the r2 score of the rf regressor is less than XGB Regressor, so with Cross Validation,

XGB is the best model, and it performs consistently well in all folds of test and train data sets.

5.2 True vs Predicted Analysis

```
PredictionErrorDisplay.from_estimator(
    xgb_regressor, X_train, y_train, kind="actual_vs_predicted", scatter_kwargs=dict(alpha=0.5)
);
```

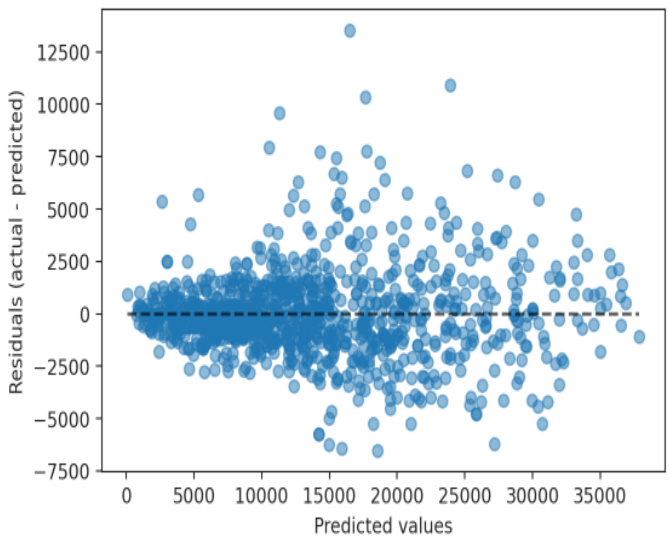


The actual vs predicted graphs of XGB regressor is plotted. Given there were no errors in the prediction made by the model, the points would have coincided and fitted in the line.

It is evident from the graph that the model is very efficient in predicting the price below 19250 and after that the variation increases.

For instance, consider a point whose actual value is more than 28625, however the predicted value is only 19250. The difference which is explained by the residual graph, the difference or residual is around 10000. The RMSE of the model is 2261.

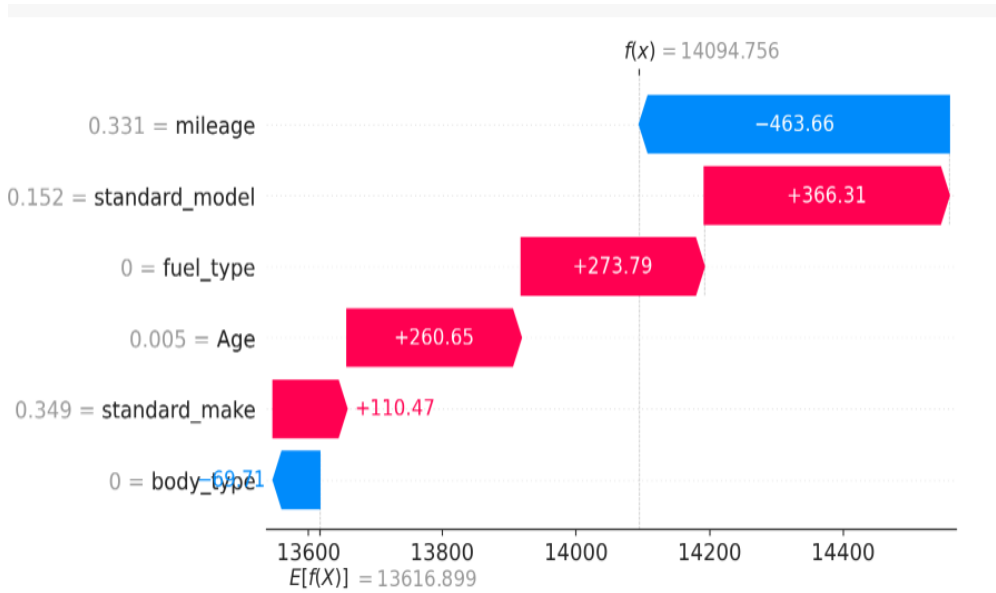
```
PredictionErrorDisplay.from_estimator(
    xgb_regressor, X_train, y_train, kind="residual_vs_predicted", scatter
);
```



A residual graph is obtained by subtracting predicted value from original true value. Ideally, the residual graph should have points scattered around the line x=0, indicating they have equal variance. The variation seems to be more after the prediction reached 15000 which means the model was able to predict consistently below 15000 and after the prediction is not that consistent. This might be because the dataset used to train the model have large number of vehicles with lower price. So, it could accurately predict lower values and not higher values.

5.3 Global and Local Explanations with SHAP

Each of the value of SHAP corresponds to the contribution of a specific feature to a model's prediction for a particular instance. In other words, they represent the impact of each feature on the model's output for that instance.

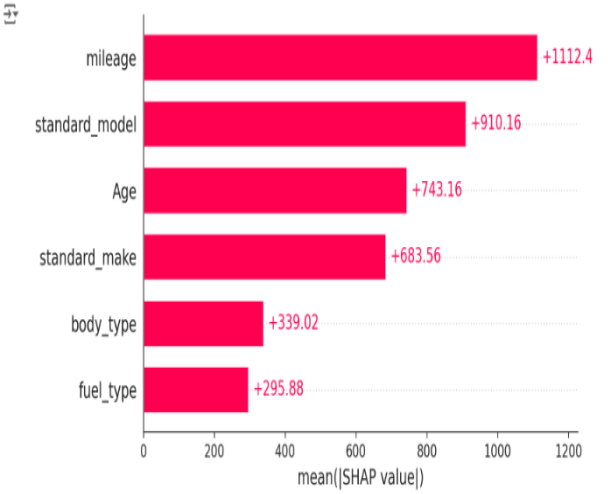


LOCAL EXPLANATION

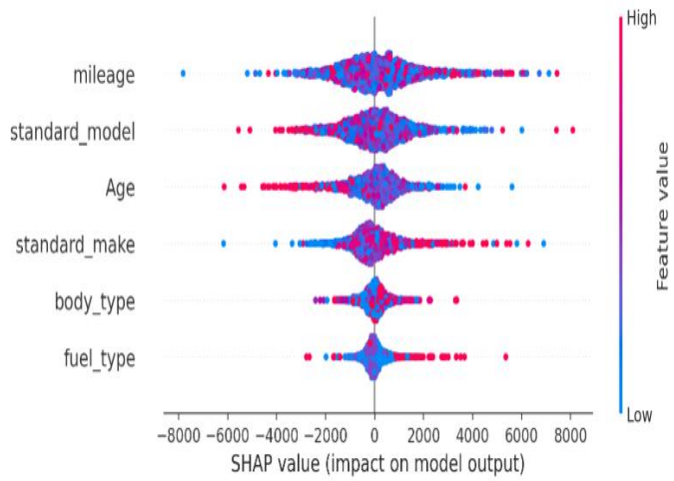
- $f(x)$ = 14094.756 represents the price predicted for a specific vehicle x and $E[f(x)]$ = 13616.899 represents the expected value of the model's output over all possible vehicles drawn from the dataset.
- The SHAP values are calculated based on this difference $f(x) - E[f(x)]$) = 477.857, and they represent the contribution of each feature to explaining this difference. Positive SHAP values indicate features that contribute to increasing the model's prediction for the vehicle x compared to the average prediction, while negative SHAP values indicate features that contribute to decreasing the prediction.
- For this vehicle, the value of mileage is negative, which means it negatively contribute to the model prediction., so is the body type. The model, make, Age and fuel type for this vehicle has a positive contribution.

GLOBAL EXPLANATION

```
shap.plots.bar(shap_values)
```




```
shap.plots.beeswarm(shap_values)
```



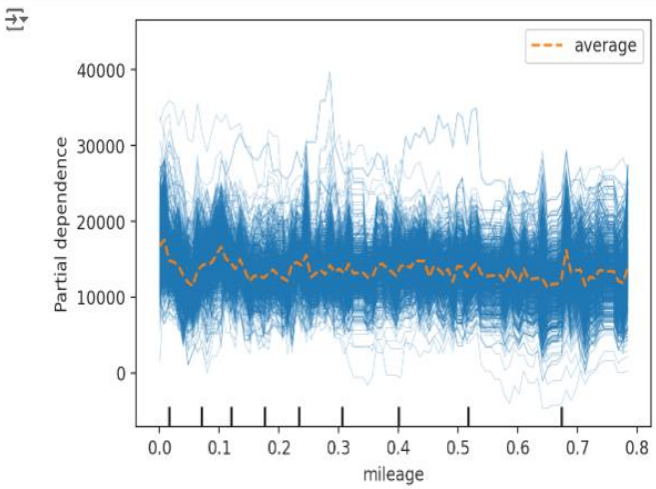
- The features mileage has large positive and negative contribution to the model as it has higher absolute mean value, thereby contributing significantly to the model prediction.
- Thus, this bar plot act as a metric form of feature importance.
- The order would be Mileage, standard model, age, standard make, body type and fuel type.

- From the bee swarm plots, it is understood that, for instance take Feature Age it has a higher SHAP values in the beginning and as the values increases, it becomes low. Which means that for cars with smaller Age has a higher value.
- For mileage, most the higher values are concentrated in the zero axis, which means that higher values are found in vehicles with shorter distance travelled.
- In case of standard model, with smaller features the SHAP values is high and as the features keep on increasing the Shap value decreased.
- An exact opposite scenario was observed for fuel type and standard make, the SHAP values is higher with larger number of features.

5.4 Partial Dependency Plots

Partial Dependency Plots (PDPs) are valuable tools for understanding the relationship between a feature and the target variable in a machine learning model while accounting for the effects of other features Each point on the plot represents the average prediction made by the model for instances with a specific value of the feature, holding all other features constant.

```
[286] PartialDependenceDisplay.from_estimator(
          xgb_regressor_scaled, X_test_scaled, features=['mileage'], kind='both'
      );
```



Nonlinearity is observed throughout the graph indicate; the feature mileage has a significant influence on models' prediction. With the increase in mileage the predicted price values are dipping in the end suggesting with higher values of mileage, price is reduced.

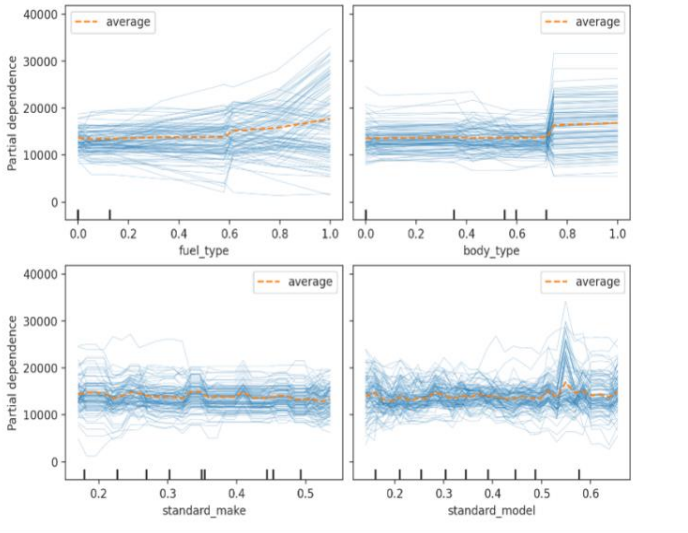
With the domain knowledge, we understand that with the increase of mileage, the price should decrease. The graph should have to be linear, in contrast, its different. This could be influenced by other factors as well. A car of maker Ferrari with higher mileage is still priced high than one from Vauxhall with shorter mileage.

The PDPs of fuel type, body type, standard make, and standard model is generated.

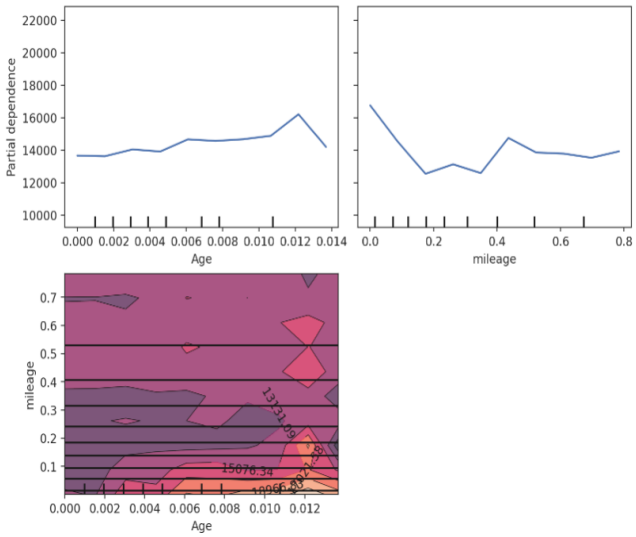
The graph corresponding to fuel types and body types has more a linear shape which depicts they do not have much influence in the prediction of the prize by the desired models. However, there is a slight increase after 0.6 and overall, it is increasing after the mid values owing to introduction of hybrid models in the model built. A similar trend is observed in body type also.

Whereas for standard make and standard model, there is irregularity through out the graph. The price of vehicle depends upon the makers. E.g., The price of Mercedes is more than that of Toyota. Also, the price could vary on the models of the same make. It depends upon the features the manufacturer built on it. For instance, BMW 5 series is expensive than BMW 3 series.

```
fig, ax = plt.subplots(figsize=(8,6), constrained_layout=True)
PartialDependenceDisplay.from_estimator(
    xgb_regressor_scaled, X_test_scaled, features=['fuel_type', 'body_type', 'standard_make', 'standard_model'],
    kind='both',
    subsample=100, grid_resolution=30, n_jobs=2, random_state=0,
    ax=ax, n_cols=2);
```




```
fig, ax = plt.subplots(figsize=(8,6), constrained_layout=True)
PartialDependenceDisplay.from_estimator(
    xgb_regressor_scaled, X_test_scaled, features=['Age', 'mileage',('Age', 'mileage') ],
    kind='average',
    subsample=100, grid_resolution=10, n_jobs=2, random_state=0,
    ax=ax, n_cols=2);
```



The ICE plots of Age suggest that with the increase of the value of Age, irregularity is observed.

A steep decrease after 0.012 indicates that price could significantly drop with the increase in Age which is justified by the domain knowledge. An increase is also obtained with increase of Age, this could be because of the introduction of Vintage cars in the model at that point.

In case of ICE of mileage, a very high value is seen in the beginning itself that portrays cars with lower mileage is predicted with higher value and with increase in mileage it is declined, however, an increase was observed afterwards may because of the interaction with other features such as Age, standard make or model.

A 2-D plot of mileage vs Age is also plotted.

The darker patch is observed in areas with lower mileage value and Age value as they are highly corelated. Even though, with the increase of age, the darker patch continued when the value of mileage was under 0.4. This indicates, mileage has greater influence on the model than the feature Age.